Практическая работа № 10

Изучение криптографических библиотек в Python

Модули, включенные в стандартную библиотеку Python, имеют довольно ограниченные возможности с точки зрения криптографии. Однако, даже они могут оказаться полезными для решения ряда задач, связанных с обеспечением безопасности данных.

Одна из таких задач - хэширование , то есть использование функции или алгоритма для сопоставления данных объекта с целочисленным значением. Результат этой функции называется хеш-значением или просто хэшем (hash). Процесс хэширования состоит из нескольких этапов:

- Определяют, целостность каких файлов нужно контролировать.
- Для каждого файла вычисляют значение его хэша по специальному алгоритму и сохраняют результат.
- Через необходимое время производят аналогичный расчёт и сравнивают результаты. Если значения отличаются, значит информация, содержащаяся в файле, была изменена.

Хэширование находит применение в разных областях, от хранения паролей до проверки целостности данных и аутентификации сообщений.

HMAC - один из механизмов проверки целостности информации, позволяющий гарантировать то, что данные, передаваемые или хранящиеся в ненадёжной среде, не были изменены посторонними лицами. Механизм HMAC использует имитовставку (MAC), описан в RFC 2104. MAC — стандарт, описывающий способ обмена данными и способ проверки целостности передаваемых данных с использованием секретного ключа. Два клиента, использующие MAC, как правило, используют общий секретный ключ. HMAC — надстройка над MAC; механизм обмена данными с использованием секретного ключа (как в MAC) и хеш-функций. В HMAC данные «смешиваются» с ключом, и хеш-функция применяется дважды.

Существуют и другие механизмы, позволяющие одновременно использовать данные и секретный ключ в существующих алгоритмах хеширования, но HMAC стал наиболее распространенным ввиду своих преимуществ:

- возможность использования хеш-функций, уже имеющихся в программном продукте;
- отсутствие необходимости внесения изменений в реализации существующих хеш-функций (внесение изменений может привести к ухудшению производительности и/или ухудшению криптостойкости);
- возможность замены хеш-функции в случае появления более безопасной или более быстрой хеш-функции.

Реализация HMAC является обязательной для протокола IPsec. HMAC используется и в других протоколах интернета, например, TLS.

1.Изучение модуля hashlib

Модуль hashlib, входящий в стандартную библиотеку Python, представляет собой модуль, содержащий интерфейс для самых популярных алгоритмов хеширования.

1.1. Импортируйте модуль hashlib с помощью команды:

import hashlib

1.2. Получите список доступных алгоритмов хэширования. Для получения такого списка используйте методы algorithms_available и algorithms_guaranteed:

print(hashlib.algorithms_available)
print(hashlib.algorithms_guaranteed)

Метод algorithms_available создает список всех алгоритмов, доступных в системе. В данном случае в списке можно заметить дубликаты названий. Метод algorithms_guaranteed перечисляет только алгоритмы модуля. Всегда присутствуют md5, sha1, sha224, sha256, sha384, sha512.

1.3. Выберите один из выведенных алгоритмов хэширования, внесите в отчет его основные характеристики.

1.4. Сформируйте текстовую строку, содержащую свои фамилию, имя и номер группы, записанные латиницей. Выведите результат хэширования этой строки по выбранному в пункте 3 алгоритму.

Пример:

```
import hashlib
hash_object = hashlib.sha256(b'Ivanov Petr BIST-28-7')
print(hash_object.hexdigest())
```

Код ниже принимает строку и выводит дайджест HEX данной строки. Метод hexdigest возвращает строку HEX, которая и представляет собой хеш для последовательности байтов. Обратите внимание, что "b" предшествует литералу строки, происходит конвертация строки в байты, оттого, что функция хеширования принимает только последовательность байтов в качестве параметра. В предыдущей версии библиотеки принимался литерал строки.

1.5. Реализуйте ввод с консоли и его хеширование по выбранному алгоритму. При этом не забудьте закодировать строку в последовательности байтов.

Пример:

```
import hashlib
mystring = input('Enter String to hash: ')
```

```
# Предположительно по умолчанию UTF-8
hash_object = hashlib.md5(mystring.encode())
print(hash_object.hexdigest())
```

Хэширование широко используется для защиты паролей, так как даже при утечке хешированных данных восстановить оригинальный пароль крайне сложно. Однако, хеширование само по себе не всегда достаточно для полной безопасности. Злоумышленники могут применять атаки типа радужных таблиц (rainbow tables) для нахождения исходных значений по их хешам.

Для усиления защиты применяется salting — добавление случайных данных к исходным паролям перед их хешированием. Этот подход значительно усложняет задачу для злоумышленников, так как каждый хеш становится уникальным, даже если несколько пользователей имеют одинаковый пароль. В результате, атаки с использованием радужных таблиц и другие методы подбора паролей становятся менее эффективными.

Одним из возможных методов защиты является использование более медленного метода хеширования. Это значит, что для вычисления множества хешей за определенный промежуток времени потребуется больше времени. Получается, поиск становится нереальном долгим, и на обнаружение совпадения может не хватить всей вашей жизни.

PBKDF2 является функцией выведения ключей, где пользователь может устанавливать вычислительные затраты. Это нужно для замедления вычисление ключа, чтобы сделать его более непрактичным для его взлома через брутфорс. Таким образом, для получения определенной длины ключа требуется пароль, соль и число или итерации, которые можно сравнить с хешем, поскольку это также односторонняя функция хеширования.

При большом числе итераций алгоритм вычисляет конечный результат дольше. Это совершенно нормально для тех, кому нужно сделать только

одну-две попыток проверки верности пароля, однако для миллиарда попыток потребуют очень много времени.

Заметим, что использование метода PBKDF2 не останавливает атаки грубой силы/перебора по словарю или использование радужных таблиц, а просто делает методы более сложными в вычислительном отношении.

PBKDF2_HMAC является реализацией функции получения ключа PBKDF2, использующей HMAC в качестве псевдослучайной функции. Функция pbkdf2_hmac присутствует в библиотеке hashlib и принимает пять параметров:

- hash_name: алгоритм хеширования для HMAC;
- password: пароль, превращенный в ключ;
- salt: случайно сгенерированная соль;
- iterations: итерации в вычислении (чем больше, тем длиннее вычисления);
- dklen: длина ключа вывода (не обязательно).

Пример:

```
import binascii # Преобразование между двоичным кодом и ASCII
import hashlib # Подключаем хэш-библиотеку
dk = hashlib.pbkdf2_hmac(hash_name='sha512',
password=b'password1234',
salt=b'salt',
iterations=10000)
result = binascii.hexlify(dk) # Возвращает 16-ричное представление
print(result)
```

В этом примере создается хэш **SHA256** в пароле при помощи некоторой соли с применением 10 000 итераций.

1.6. Задайте пароль в виде строки длиной не менее 12 символов и произвольное значение «соли». Используя выбранный в пункте 1.3 алгоритм хэширования, реализуйте функцию получения ключа PBKDF2, использующей HMAC в качестве псевдослучайной функции. Значения параметров iterations и dklen задайте самостоятельно.

1.7. Повторите пункт 1.6 сначала с увеличенным значением числа итераций, затем с увеличенным значением параметра dklen. Сделайте выводы.

2. Изучение симметричного шифрования в библиотеке cryptography

Cryptography — это библиотека Python для обеспечения криптографических функций. Библиотека предоставляет разнообразные инструменты и алгоритмы для выполнения шифрования, дешифрования, создания хешей сообщений и генерации ключей. Она является надежным решением для реализации различных методов шифрования, начиная от симметричного и асимметричного шифрования, и заканчивая созданием цифровых подписей и хешей.

Возможности Cryptography:

- Симметричное шифрование (AES, Camellia, Blowfish и др.) — позволяет зашифровать и расшифровать данные с использованием одного ключа.
- Асимметричное шифрование (RSA, ECC и др.) — использует публичный и приватный ключи для шифрования и подписи сообщений.
- Генерация ключей и ключевых пар для разных алгоритмов.
- Работа с сертификатами X.509.
- Вычисление и проверка цифровых подписей и хешей.
- Управление процессом шифрования (режимы работы, векторы инициализации).
- Интеграция с системами управления ключами и хранилищами сертификатов.
- Поддержка стандартов шифрования как для веб (TLS, SSL), так и для других протоколов.

2.1. Установите библиотеку Cryptography, используя команду:

pip install cryptography

2.2. Импортируйте из библиотеки Cryptography модуль Fernet и сгенерируйте с его помощью симметричный ключ шифрования:

```
from cryptography.fernet import Fernet
cipher_key = Fernet.generate_key()
print(cipher_key)
```

2.3. Зашифруйте текстовую строку, сформированную в пункте 1.4, применяя сгенерированный в пункте 2.2 симметричный ключ с использованием метода encrypt() :

```
cipher = Fernet(cipher_key)
text = b'Ivanov Petr BIST-28-7'
encrypted_text = cipher.encrypt(text)
print(encrypted_text)
```

2.4. Расшифруйте зашифрованную строку с использованием метода decrypt() и убедитесь в правильности расшифровки:

```
decrypted_text = cipher.decrypt(encrypted_text)
print(decrypted_text)
```

3. Изучение асимметричного шифрования в библиотеке cryptography

Теперь перейдем к рассмотрению асимметричной криптографии в python. Для этого используем такой раздел библиотеки cryptography как hazmat (Harzardous Material).

Совместно с алгоритмами ассиметричного шифрования, как правило, используются дополнения, которые служат для повышения криптостойкости. Так, например, довольно часто применяется OAEP (оптимальное асимметричное шифрование с дополнением).

Классическая схема OAEP представляет собой двухъячеечную сеть Фейстеля, где в каждой ячейке данные преобразуются с помощью криптографической хеш-функции. На вход сеть получает сообщение с дописанными к нему проверяющими нулями и ключ — случайную строку.

Алгоритм OAEP применяется для предварительной обработки сообщения перед использованием RSA. Сначала сообщение дополняется до фиксированной длины с помощью OAEP, затем шифруется с помощью RSA. Совместно, такая схема шифрования получила название RSA-OAEP и является частью действующего стандарта шифрования с открытым ключом (RFC 3447).

3.1. Сгенерируйте пару ключей RSA (открытый и закрытый ключи), которая будет использоваться для шифрования и дешифрования. Рекомендуемый размер ключа — 2048 бит или выше:

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.backends import default_backend
def generate_rsa_key_pair(key_size):
    private_key = rsa.generate_private_key(public_exponent=65537,
    key_size=key_size,
    backend=default_backend())
    public_key = private_key.public_key()
    return private_key, public_key
private_key, public_key = generate_rsa_key_pair(2048)
```

3.2. Произведите сериализацию ключей RSA. Цель этой операции – возможность безопасно сохранять или передавать ключи. Рекомендуется использовать формат **PEM** и шифрование закрытого ключа с помощью парольной фразы:

```
def serialize_private_key(private_key, passphrase):
    return rivate_key.private_bytes(encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.BestAvailableEncryption(passphras
    e.encode("utf-8")))
def serialize_public_key(public_key):
    return public_key.public_bytes(encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo)
```

3.3. Сохраните сериализованные ключи:

```
with open("private_key.pem", "wb") as f:
        f.write(serialize_private_key(private_key, "your_passphrase"))
with open("public_key.pem", "wb") as f:
        f.write(serialize_public_key(public_key))
```

3.4. Создайте функции загрузки сериализованных ключей RSA обратно из файлов:

```
def load_private_key(file_path, passphrase):
    with open(file_path, "rb") as f:
            return serialization.load_pem_private_key(f.read(),
password=passphrase.encode("utf-8"), backend=default_backend())
def load_public_key(file_path):
     with open(file_path, "rb") as f:
             return serialization.load_pem_public_key(f.read(), backend=default_
backend())
```

3.5. Загрузите сериализованные ключи RSA:

```
private_key = load_private_key("private_key.pem", "your_passphrase")
public_key = load_public_key("public_key.pem")
```

3.6. Задать функции шифрования и дешифрования с помощью RSA, используя методы encrypt и decrypt соответственно:

```
    def rsa_encrypt(plaintext, public_key):
            plaintext = plaintext.encode("utf-8")
            ciphertext = public_key.encrypt(
            plaintext,
            asymmetric_padding.OAEP(
            mgf=asymmetric_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None))
            return ciphertext
     def rsa_decrypt(ciphertext, private_key):
            plaintext = private_key.decrypt(
            ciphertext,
            asymmetric_padding.OAEP(
            mgf=asymmetric_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None))
            return plaintext.decode("utf-8")
```

3.7. Зашифруйте текстовую строку, сформированную в пункте 1.4, применяя загруженный в пункте 3.5 публичный ключ:

```
message = "Ivanov Petr BIST-28-7"
encrypted_message = rsa_encrypt(message, public_key)
print("Encrypted message:", encrypted_message)
```

3.8. Расшифруйте результат выполнения пункта 3.7, применяя загруженный в пункте 3.5 приватный ключ:

```
decrypted_message = rsa_decrypt(encrypted_message, private_key)
print("Decrypted message:", decrypted_message)
```

Сравните исходную и расшифрованную строку.

4. Изучение инфраструктуры открытых ключей

Существует стандарт, описывающий форматы сертифицированных открытых ключей (“цифровых сертификатов”) – X.509. Соответствие формата данному стандарту позволяет осуществлять обмен защищенными сообщениями между пользователями средств защиты информации различных производителей (но в рамках определенных алгоритмов шифрования и ЭП).

Минимальный набор полей формата сертифицированного открытого ключа, в соответствии с версией 1 стандарта X.509, выглядит так:

- Номер версии стандарта, которой соответствует ключ. На данный момент существует 3 версии стандарта.
- Серийный номер ключа. Уникальный номер, присваиваемый конкретному ключу.
- Код алгоритма ЭП, в котором используется конкретный ключ.
- Наименование центра генерации ключей, выпустившего данный ключ.
- Срок действия ключа. Указывается диапазон, т.е. дата начала и дата окончания.
- Имя абонента, которому принадлежит ключ.
- Сертифицирующая подпись данного открытого ключа, поставленная центром сертификации ключей.

Версия 2 стандарта X.509 содержит следующие дополнительные поля:

- Идентификатор центра генерации ключей.
- Идентификатор абонента.

Версия 3 стандарта X.509 разрешает записывать в ключ дополнительную информацию, специфичную для конкретного средства защиты, и устанавливает формат дополнительных записей.

Следующий шаг на пути стандартизации обмена защищенными сообщениями – определение инфраструктуры открытых ключей (PKI – Public Key Infrastructure). PKI представляет собой набор агентов и правил, предназначенных для управления ключами, политикой безопасности и собственно обмена защищенными сообщениями. Структура PKI приведена на рисунке 1. В PKI используются ключи, соответствующие стандарту X.509.

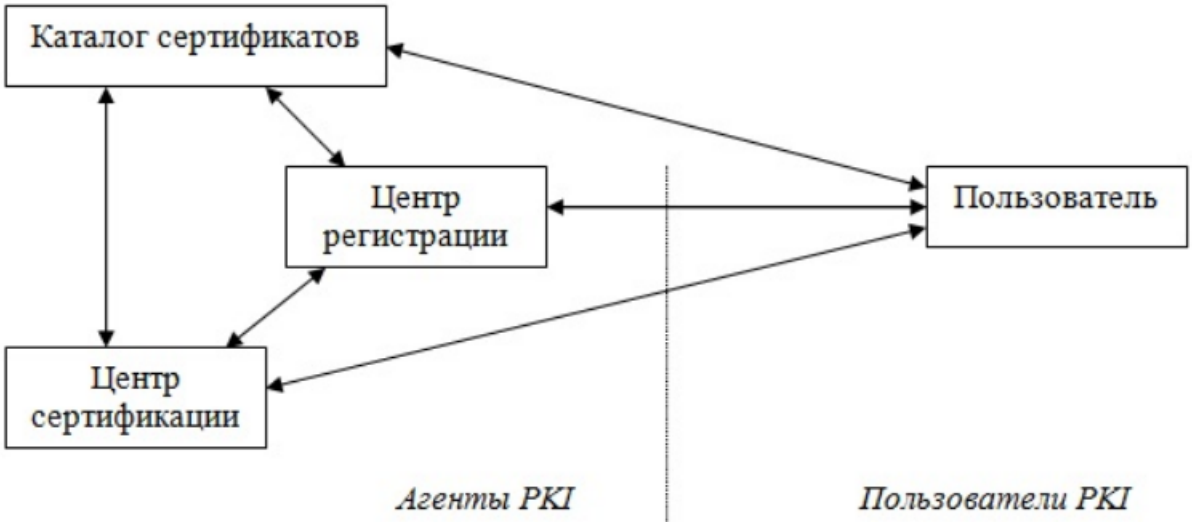

Рис.1 Структура PKI

PKI обеспечивает обработку используемых ключей и идентификацию их владельцев при использовании различных криптографических протоколов.

Основные компоненты PKI:

- Центр сертификации (CA) — удостоверяющий орган, выпускающий цифровые сертификаты.
- Сертификат — электронный документ, связывающий открытый ключ с идентификатором владельца.
- Самоподписанный сертификат — сертификат, подписанный своим же создателем (не доверенным CA).

4.1. Импортируйте необходимые библиотеки для Python-скрипта, генерирующего самоподписанный сертификат X.509 с использованием библиотеки cryptography:

```
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.backends import default_backend
from datetime import datetime, timedelta
```

4.2. Сгенерируйте приватный ключ:

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
    backend=default_backend()
)
```

4.3. Задайте информацию о владельце сертификата:

```
subject = issuer = x509.Name([
    x509.NameAttribute(NameOID.COUNTRY_NAME, "RU"),
    x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME,
"Moscow"),
    x509.NameAttribute(NameOID.LOCALITY_NAME, "Moscow"),
    x509.NameAttribute(NameOID.ORGANIZATION_NAME,          "Test
Company"),
    x509.NameAttribute(NameOID.COMMON_NAME, "localhost"),
])
```

4.4. Создайте самоподписанный сертификат

```
cert = x509.CertificateBuilder().subject_name(
    subject
).issuer_name(
    issuer
).public_key(
    private_key.public_key()
).serial_number(
    x509.random_serial_number()
).not_valid_before(
    datetime.utcnow()
```

```
).not_valid_after(
    datetime.utcnow() + timedelta(days=365)
).add_extension(
    x509.BasicConstraints(ca=True, path_length=None),
    critical=True,
).sign(private_key, hashes.SHA256(), default_backend())
```

4.5. Сохранените сертификат и ключ в файлы

```
with open("self_signed.crt", "wb") as cert_file:
    cert_file.write(cert.public_bytes(serialization.Encoding.PEM))

with open("private_key.key", "wb") as key_file:
    key_file.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption(),
    ))

print("Самоподписанный сертификат и приватный ключ созданы!")
```

4.6. Откройте файл self_signed.crt в текстовом редакторе — он должен содержать данные в формате PEM. Приведите скриншот сертификата в отчете.

5. Изучение методов обфускации кода с использованием Python

Обфускация — это процесс намеренного усложнения исходного кода с целью затруднения его анализа, реверс-инжиниринга и модификации. Применяется как в легальном ПО (защита коммерческих продуктов), так и в вредоносном (сокрытие функциональности).

Основные методы обфускации:

- Переименование идентификаторов — замена имен переменных, функций и классов на бессмысленные последовательности символов.
- Шифрование строк — кодирование строковых литералов с последующей расшифровкой во время выполнения.
- Минификация — удаление комментариев, пробелов и сокращение имен переменных.

- Добавление ложной логики — вставка неисполняемого или бесполезного кода.
- Использование динамического выполнения — генерация и выполнение кода через eval() или exec().

Наиболее распространенные библиотеки Python для обфускации:

- pyminifier — минификация и базовое переименование переменных.
- pyobfuscate — более сложные методы обфускации, включая шифрование строк.
- astor + ast — модификация абстрактного синтаксического дерева (AST) для глубокой обфускации.
- obfuscator-io (онлайн-сервис) — автоматическая обфускация Python-скриптов.

Кроме того, применяются инструменты командной строки, такие, как Pyarmor — средство для обфускации скриптов Python, привязки обфусцированных скриптов к определенным машинам и установки сроков действия для обфусцированных скриптов. Его основные характеристики:

- Простая замена: запутанные скрипты остаются в виде стандартных файлов *.py* , что позволяет им в большинстве случаев легко заменять исходные скрипты Python.
- Сбалансированная обфускация: предлагает несколько способов обфускации скриптов для обеспечения баланса безопасности и производительности.
- Необратимая обфускация: переименовывает функции, методы, классы, переменные и аргументы.
- Преобразование функций C: преобразует некоторые функции Python в функции C и компилирует их в машинные инструкции, используя высокие параметры оптимизации для необратимой обфускации.

- Привязка скриптов: привязывает запутанные скрипты к определенным машинам или устанавливает даты истечения срока действия для запутанных скриптов.
- Защита Themida: защищает запутанные скрипты с помощью Themida (только для Windows).

5.1. Установите библиотеку pyarmor:

pip install pyarmor

5.2. Создайте рабочую папку и подготовьте в этой папке тестовый файл (example.py). Это должна быть небольшая программа на python, например, выводящая в командной строке Ваши имя, фамилию и номер группы:

print("Ivanov Petr BIST-28-7")
input()

5.3. Откройте командную строку и перейдите с помощью команды **cd** в рабочую папку с исходным кодом. Проверьте работоспособность кода командой:

python example.py

5.4. Обфусцируйте код командой:

pyarmor gen example.py

5.5. Убедитесь, что после успешного выполнения команды в рабочей папке была создана другая папка с названием **dist.** В ней должен находиться файл с таким же названием: **example.py**, но уже содержащий обфусцированный код.

5.6. Проверьте работоспособность обфусцированного кода. Для этого перейдите командой **cd** в папку **dist** и запустите python-файл командой:

python example.py

Убедитесь, что файл выводит тот же текст в командную строку.

5.7. Соберите обфусцированный код в исполняемый файл с расширением **exe** с помощью команды:

pyinstaller – F example.py

5.8. После успешного завершения компиляции обфусцированного кода необходимо перейти в следующую папку dist и в ней найти программу **example.exe**. Запустив эту программу на выполнение, убедитесь в ее работоспособности.

Примечание: Данная работа носит учебный характер и не должна использоваться для создания вредоносного ПО.

6. Ответьте письменно на один контрольный вопрос из таблицы 1, соответствующий Вашему номеру по списку в журнале.

Таблица 1

Номер по журналу	Контрольный вопрос
1, 17	Какие алгоритмы хэширования используются в модуле hashlib?
2, 18	Какие алгоритмы хэширования на сегодняшний день считаются устаревшими и не рекомендованы к применению?
3, 19	Для чего «солят» хэш-функции?
4, 20	Поясните, какие механизмы повышения криптостойкости использует стандарт PBKDF2 ?
5, 21	Что такое HMAC и для каких задач применяется?
6, 22	Для чего применяется OAEP?
7, 23	Что такое сериализация ключей? Для чего она применяется?
8, 24	Каков минимальный набор полей формата сертифицированного открытого ключа в стандарте X.509 ?
9, 25	Что такое PKI ? Каковы её преимущества и недостатки?
10, 26	Что такое самоподписанный сертификат? Каковы его преимущества и недостатки?
11, 27	В чем заключается особенность формата PEM? Для каких задач он был разработан и для каких задач используется в настоящее время?
12, 28	Что такое обфускация кода?
13, 29	Какие задачи решает обфускация?
14, 30	Какие виды обфускации Вам известны?
15, 31	Какие библиотеки Python для обфускации Вы знаете?
16, 32	Возможна ли обфускация кода средствами командной строки?